

T3 Stack + tRPC

Stack moderno para aplicaciones Next.js full stack con tipado de extremo a extremo

1. T3 Stack

Es una combinación de tecnologías para Next.js elegidas para trabajar muy bien juntas, creada y popularizada por **Theo (t3.gg)**. Está obsesionado con dos cosas: **typesafety** y **simplicidad**.

Qué incluye

Tecnología	Para qué
Next.js	Framework base
TypeScript	Tipado en todo el stack
Tailwind CSS	Estilos
tRPC	Comunicación cliente-servidor tipada
Prisma	Base de datos (ORM)
NextAuth.js	Autenticación

No tienes que usar todas — puedes elegir cuáles incluir al crear el proyecto.

Crear un proyecto

```
npm create t3-app@latest
```

Te pregunta qué partes quieres incluir y monta todo configurado automáticamente.

Relación con tu stack actual

Si ya sabes Next.js + TypeScript + Tailwind, el T3 Stack te añade básicamente **tRPC + Prisma + NextAuth**. No es un salto enorme, es sumar piezas encima de lo que ya tienes.

¿Cuándo usarlo?

- Una app full stack en Next.js con todo bien integrado desde el principio
- Máxima seguridad de tipos de extremo a extremo
- No perder tiempo configurando cada pieza por separado

2. tRPC

Librería que permite **llamar funciones del servidor desde el cliente con tipado completo**, sin escribir una API REST ni GraphQL. El nombre viene de **TypeSafe Remote Procedure Call**.

El problema que resuelve

En una app Next.js normal defines los tipos dos veces (servidor y cliente) y si cambias algo en el servidor tienes que acordarte de actualizarlo también en el cliente. Si te olvidas, TypeScript no te avisa — el error solo aparece

en ejecución.

Sin tRPC — tipos definidos dos veces:

```
// servidor: defines el endpoint
export async function GET() {
  return Response.json(await db.getUsuarios())
}

// types: defines los tipos manualmente
interface Usuario { id: number; nombre: string; email: string }

// cliente: haces el fetch con los tipos
fetch('/api/usuarios').then((res) => res.json() as Usuario[])
```

Con tRPC — una sola definición, tipo fluye automáticamente:

```
// servidor
const appRouter = router({
  getUsuarios: procedure.query(async () => {
    return await db.getUsuarios() // TypeScript infiere el tipo
  }),
})

// cliente
const { data: usuarios } = trpc.getUsuarios.useQuery()
// TypeScript ya sabe que usuarios tiene el tipo correcto
```

Los dos tipos de operaciones

query — para leer datos (equivale a GET):

```
trpc.getUsuario.useQuery({ id: '123' })
trpc.getProductos.useQuery()
```

mutation — para escribir datos (equivale a POST/PUT/DELETE):

```
const crear = trpc.crearUsuario.useMutation()
crear.mutate({ nombre: 'Ana', email: 'ana@email.com' })
```

Configuración en Next.js — paso a paso

1. Instalar dependencias:

```
npm install @trpc/server @trpc/client @trpc/react-query @tanstack/react-query zod
```

2. Inicializar tRPC:

```
// server/trpc.ts
import { initTRPC } from '@trpc/server'

const t = initTRPC.create()
export const router = t.router
export const procedure = t.procedure
```

3. Crear el router con las rutas:

```
// server/router.ts
import { router, procedure } from '../trpc'
import { z } from 'zod'

export const appRouter = router({
  saludo: procedure
    .input(z.object({ nombre: z.string() }))
    .query(({ input }) => `Hola, ${input.nombre}`),
})

export type AppRouter = typeof appRouter
```

4. Conectar con Next.js:

```
// app/api/trpc/[trpc]/route.ts
import { fetchRequestHandler } from '@trpc/server/adapters/fetch'
import { appRouter } from '@server/router'

const handler = (req: Request) =>
  fetchRequestHandler({
    endpoint: '/api/trpc',
    req,
    router: appRouter,
    createContext: () => ({}),
  })

export { handler as GET, handler as POST }
```

5. Configurar el cliente:

```
// utils/trpc.ts
import { createTRPCReact } from '@trpc/react-query'
import type { AppRouter } from '@server/router'

export const trpc = createTRPCReact<AppRouter>()
```

6. Usar en un componente:

```
'use client'
import { trpc } from '@utils/trpc'

export default function Saludo() {
  const { data } = trpc.saludo.useQuery({ nombre: 'Carla' })
  return <p>{data}</p> // → 'Hola, Carla'
}
```

Comparativa REST vs tRPC

	REST API	tRPC
Definir tipos	Dos veces	Una vez
Errores de tipo	En ejecución	Al compilar
Documentación	Manual o Swagger	Automática
Curva de aprendizaje	Baja	Media
APIs públicas	Ideal	No recomendado
Apps full stack	Funciona	Ideal

¿Cuándo NO usar tRPC?

- Si tu API la van a consumir **terceros** — en ese caso REST o GraphQL son mejor opción
- Si ya tienes una API REST montada — no vale la pena migrar